

Prog 4 – ZH-2

A feladatsort figyelmesen olvasd el!

A ZH-t zip fájlként kell feltölteni az alábbi formátumban: `név_neptun_kód_prog4_zh2.zip`

Az alkalmazáserver helye a labor gépein: <C:\apache-tomcat>

A ZH feladat leírása és kiinduló projektje mellett található egy **auditalo.exe** állomány, amit első lépésként tölts le.

Hozz létre egy mappát az alábbi néven: `<név_neptun_kód>_prog4_zh2`

Ebbe a létrehozott mappába másold be az **auditalo.exe** állományt, majd indítsd el, ez létre fog hozni két mappát: **zh**, **zh_audit**

Ha a Windows blokkolná a futtatást, akkor kattints a felugró ablakban a **További információ** linkre, majd a **Futtatás mindenképpen** opcióra, ezután megnyílik egy command window, **ezt ne zárd be, a teljes ZH alatt futnia kell.**

Ezt követően tölts le a kiinduló projekteket, majd csomagold ki őket (vagy másold át a zh2-feladat és a zh2-ws-feladat mappákat) a **zh** mappába és itt dolgozz (innen nyisd meg őket Intellij Idea-ban). A **zh_audit** mappa tartalmához TILOS nyúlni és annak módosítása is TILOS.

Nem forduló kód (maven build használatával) vagy el nem induló alkalmazás automatikusan 0 pontos ZH-t eredményez.

ChatGPT (vagy bármilyen AI) és chat alkalmazások megnyitása vagy használata, illetve egymás segítése tilos, ennek észrevétele (akár a ZH során, akár a beadott megoldás alapján) szintén 0 pontos ZH-t eredményez.

Az auditáló futtatásának hiánya, annak idő előtti leállítása, a fájljainak módosítása 0 pontos ZH-t eredményez.

Órai anyag és internet használható a feladatok megoldása során, viszont teljes blokkok másolása esetén az adott feladat 0 pontot ér.

A 0. feladat git kezeléshez fog kapcsolódni, az ott leírtak beugrónak számítanak.

Git repo hiánya esetén a ZH 0 pontos, commitálás nélkül beadott kód 0 pontot ér.

A teljes ZH egyben történő commitálása esetén a ZH szintén 0 pontos.

Több feladat egyben történő commitálása esetén csak az egyik feladat lesz figyelembe véve a javítás során.

Mivel a ZH jelentős része kapcsolódik az adatbázishoz és az adatok is ott kerülnek tárolásra, így az adatbázis kapcsolat hiánya esetén a ZH 0 pontot ér.

A **zh** mappán kívüli található megoldások nem lesznek figyelembe véve, arra 0 pont jár.

Beadáskor egyetlen tömörített (ZIP) állományt kell feltölteni. Ehhez a korábban létrehozott `<név_neptun_kód>_prog4_zh2` mappát tömörítsd. A feltöltött állománynak tartalmaznia kell a **zh és **zh_udit** mappákat, valamint az auditáló programot.**

Ellenőrizd, hogy a mariadb-java-client-3.1.3.jar megtalálható-e a tomcat alatt, ha nem akkor a gyakorlaton mutatott módon töltsd le az internetről és helyezd el a megfelelő könyvtárban.

A feladathoz szükség van egy adatbázisra. Amennyiben elindítod a projektben található compose-t, akkor elindul az adatbázis és minden szükséges objektum létrejön benne.

Az adatbázis kapcsolathoz szükséges resource definíciója, **amit szükséges kiegészíteni a hiányzó értékekkel** (ha már van másik resource felvéve azt töröld ki):

```
<Resource name="jdbc/ZH2_2026_MariaDB" type="javax.sql.DataSource"
driverClassName="org.mariadb.jdbc.Driver"
factory="org.apache.tomcat.jdbc.pool.DataSourceFactory"
url="jdbc:mariadb://localhost:3306/zh2_2026_database" username="" password=""
maxActive="20" maxIdle="10" maxWait="-1" />
```

Az adatbázisban két felhasználó van.

user1: neki user jogosultsága van

user2: neki user és dealer jogosultsága van

Mind a kettőnek "Password111" a jelszava. A jelszavak Bcrypttel vannak elforgatva.

Git (zh2-feladat és zh2-ws-feladat)

- Hozzon létre egy git repository-t az auditáló által létrehozott zh mappában.
- Készíts egy develop branchet, majd ezen a branchen dolgozz.
- A kapott zh2-feladat projekt mappát (ne csak a tartalmát) másold be a zh mappába és commitáld el egyben a develop branchen.
- Minden feladat megoldását külön-külön commitáld el, figyelj a beszédes commit message-kre.

Az alkalmazásunk autók adatait tárolja, jeleníti meg és ad lehetőséget a lista bővítésére. Az autókra vonatkozóan webservice-n keresztül kérdez le egyéb adatokat.

Az alkalmazásunk rest api endpointokat biztosít, hogy külső alkalmazás tudjon az autókról adatokat lekérdezni és adatokat beküldeni.

A webservice által visszaadott adatok lekérdezhetőségét rest api-n keresztül biztosítva az alkalmazás, mind a felület, mind egy külsős alkalmazás számára (a rest apit hívó külső alkalmazás elkészítése NEM része a feladatnak).

Az alkalmazásunkat security védi, hogy illetéktelenek ne tudjanak hozzáférni.

Adatbázis

1. Implementáld, illetve hozd létre a következő adat elérési osztályokat és a bennük lévő metódusokat, amik a felületek, rest api endpointok és security számára biztosítják az adatokat:

Repository, CarRepository, RoleRepository, UserRepository

SOAP

A zh2-ws-feladat webes alkalmazás webservice-en keresztül szolgáltat adatokat egy-egy adott autóról. A webservice adatokat vár a kérésben egy autóról, amik alapján egyéb adatokat ad vissza válaszként az adott autóra vonatkozóan.

Ezeket a visszakapott adatokat majd rest api-n keresztül lehet lekérdezni, mind a felületen található „Egyéb adatok” gombbal, (hogya a felhasználó láthassa az autóra vonatkozó egyéb adatokat a felületen egy felugró ablakban), mind egy külső alkalmazás számára (a rest apit hívó külső alkalmazás elkészítése NEM része a feladatnak), hogy ő is megkaphassa a webservice által visszaadott adatokat.

2. A zh2-ws-feladat projektben található CarService és SoapCarService osztályok hiányzó részeit egészítsd ki, majd a két osztály alapján generálj WSDL-t az src/main/resources/wsdl mappába (cxf-java2ws-plugin).

Az alkalmazás és a web service a 8081-es porton legyen elérhető webes alkalmazás legyen. (Az „IntelliJ Idea” Tomcat konfiguráció „Server” tab-ján a „Tomcat Server Settings” alatt található JMX portot állítsa át 1098-ra.)

3. A kigenerált WSDL-eket másold át a zh2-feladat projekt src/main/resources/wsdl mappába és generálj a zh2-feladat projektben egy klienst ehhez a web service-hez. Ezeket az osztályokat közvetlenül a target/generated mappába generáld ki (jaxws-maven-plugin).

Az zh2-feladat alkalmazás a 8080-as porton legyen elérhető.

4. A CarService osztálynak implementáld a getCarData metódusát. A kigenerált WS klienst használva hívd meg a zh2-ws-feladat projektben elkészített web service-t.

A getCarData metódust rest api endpointon keresztül hívva fogjuk szolgáltatni a webservice által visszaadott adatokat a felület és a külső alkalmazás felé.

REST

5. Hozd létre és implementáld a CarController osztályt, mint REST controller osztály, amivel a külső alkalmazás tud kommunikálni.

Az api JSON-ökkel kommunikál.

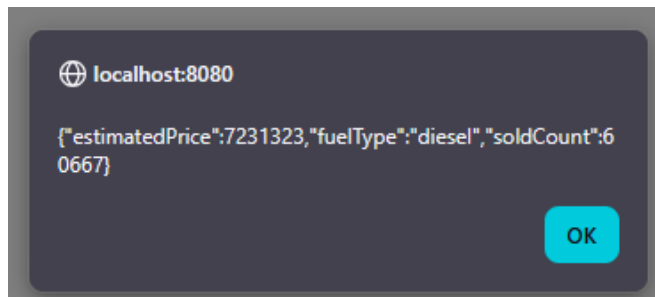
Hozz benne létre methodokat, amik meghívják minden a CarService osztályban található funkciót.

A save service metódus végzi a create és update műveleteket, a service metódus működést figyelembe véve hívd a create és update api-kból.

6. A carList.jsp oldalon található „Egyéb adatok” gombot implementáld.

Hívja meg JavaScript segítségével a fentebb implementált REST endpointok közül azt, amelyik a WS-t hívó servicet hívja.

Akár sikeres volt a hívás, akár nem, a böngésző dobjon róla egy értesítést.
Sikeres hívás esetén a WS által visszaadott adatokat jelenítse meg a felületen.



Security (zh2-feladat projekt)

7. A login.html állományt hozd létre és készítsd el a bejelentkező formot.

8. Implementáld az AuthenticationModule osztálynak a metódusait úgy, hogy az adatbázisból authenticáljon a rendszer. Ehhez használd a korábban elkészített UserRepository és a RoleRepository osztályokat.

9. Konfiguráld be az authenticálást a különböző állományokban a következőképpen:

- Legyen FORM alapú bejelentkezés.
- Az index.jsp-t el lehessen érni bejelentkezés nélkül.
- Az összes többi oldal eléréséhez be kell jelentkezni és a “user” role-hoz legyen kötve.
- Az új autó rögzítés a “dealer” role-hoz legyen kötve.
- A bejelentkező oldal legyen a login.html.
- Hibás bejelentkezés esetén irányítson át a login_failed.html-re.

10. Implementáld a kijelentkezést is a /logout pathra. Ha sikeres volt a kijelentkezés irányítsa át a rendszer a felhasználót a context path-re.